COS214 PROJECT

The Seven Deadly Syntaxers

https://github.com/elme21/COS-214-Prac-6

University of Pretoria

CONTENT

- System Overview
- Workflow Execution
- Sequence Diagrams
- Class Diagram
- Design Patterns
- Frontend
- Live Demo
- Testing
- Github
- Docker

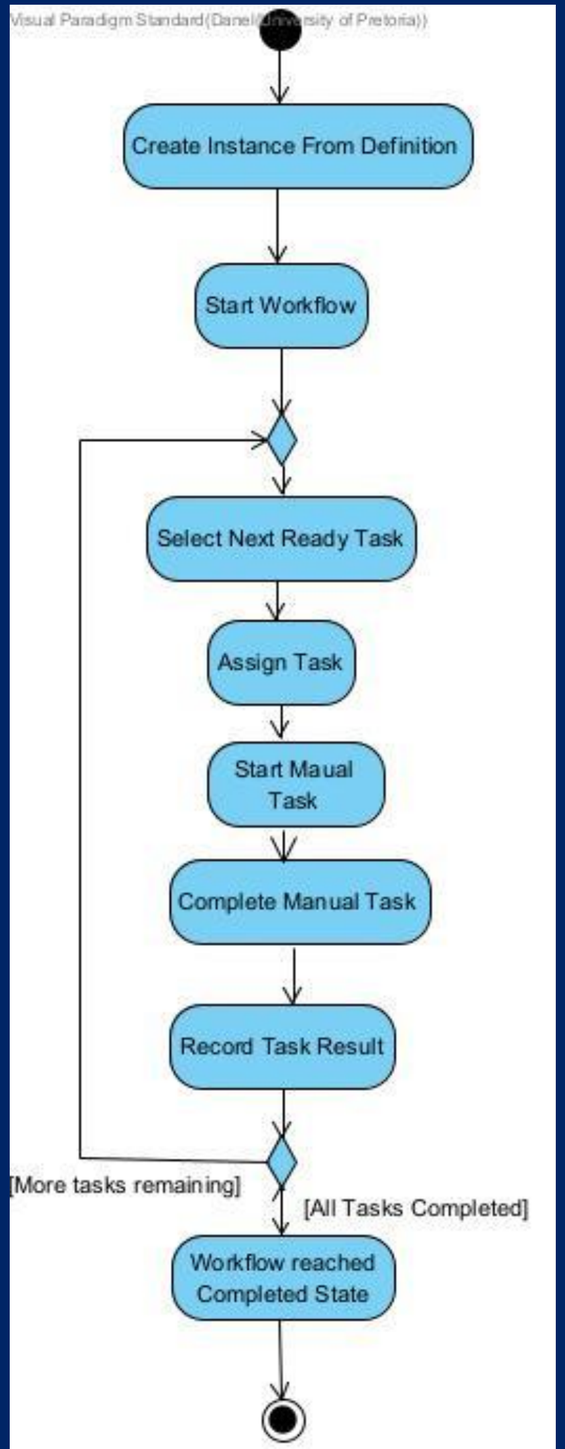
SYSTEM OVERVIEW

TIVIDY

Generic workflow management system

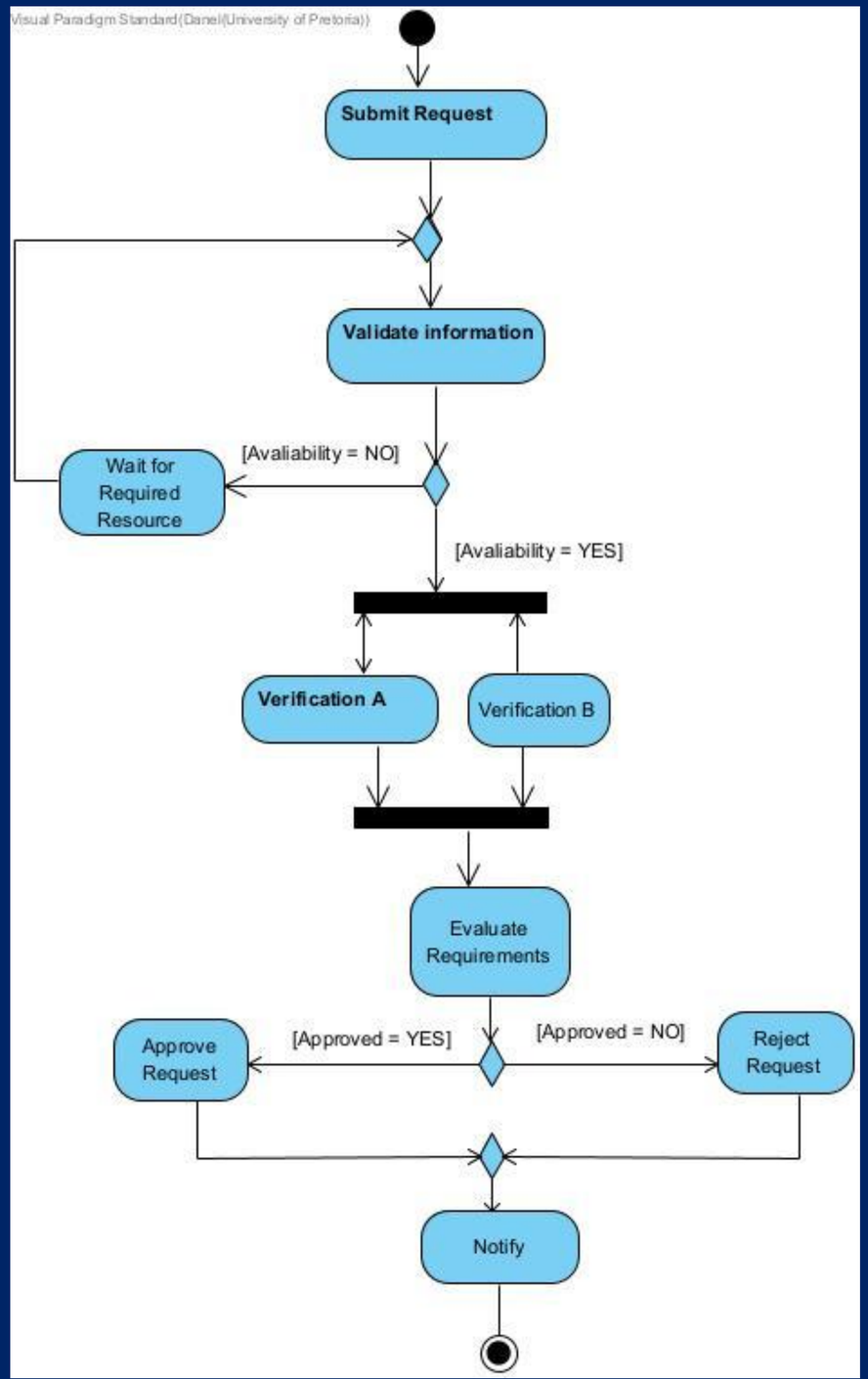
Customisable tasks, stages and dependencie

Reusable workflow definitions



WORKFLOW EXECUTION

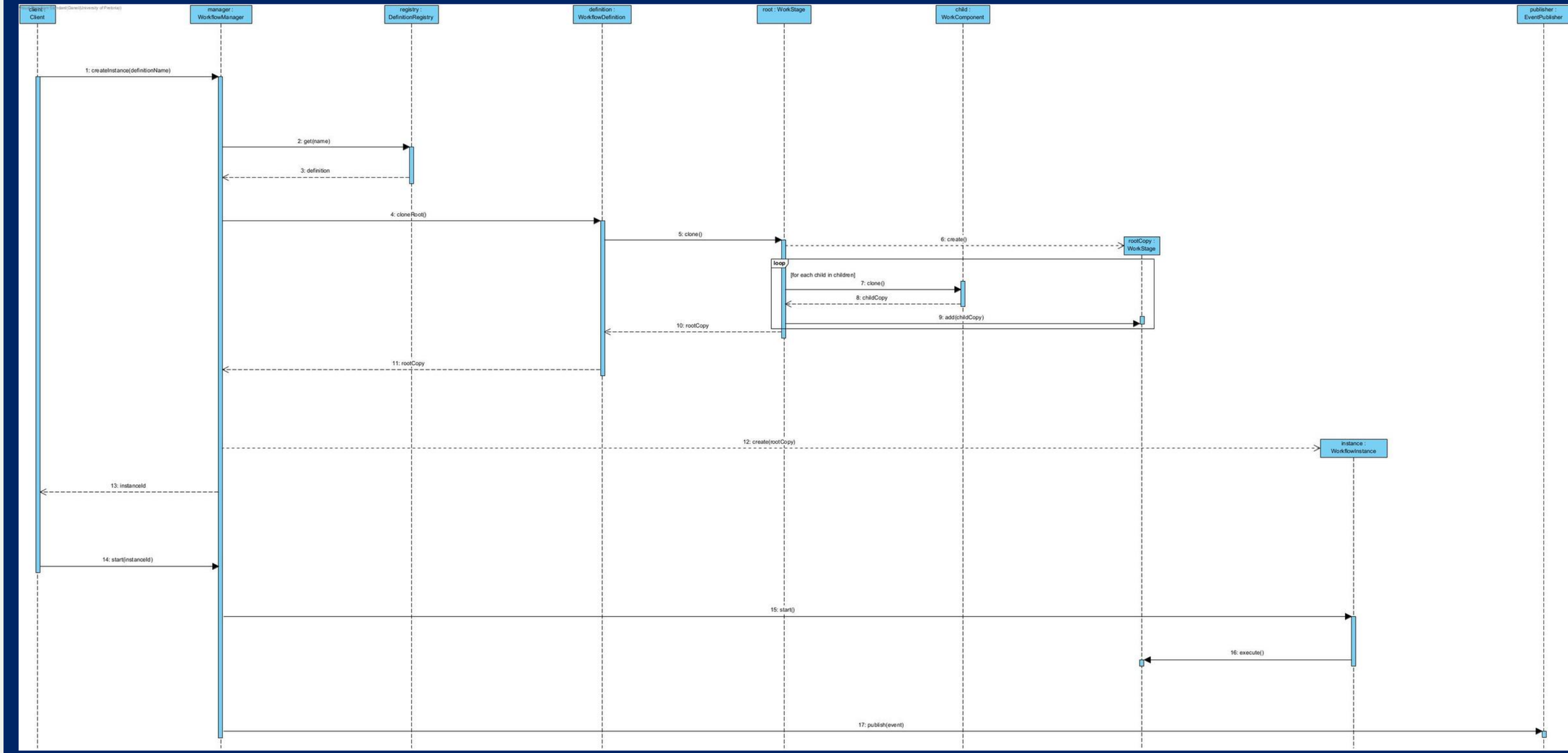
- Create a workflow instance
- Assign, start and complete tasks
- Repeat until workflow completion



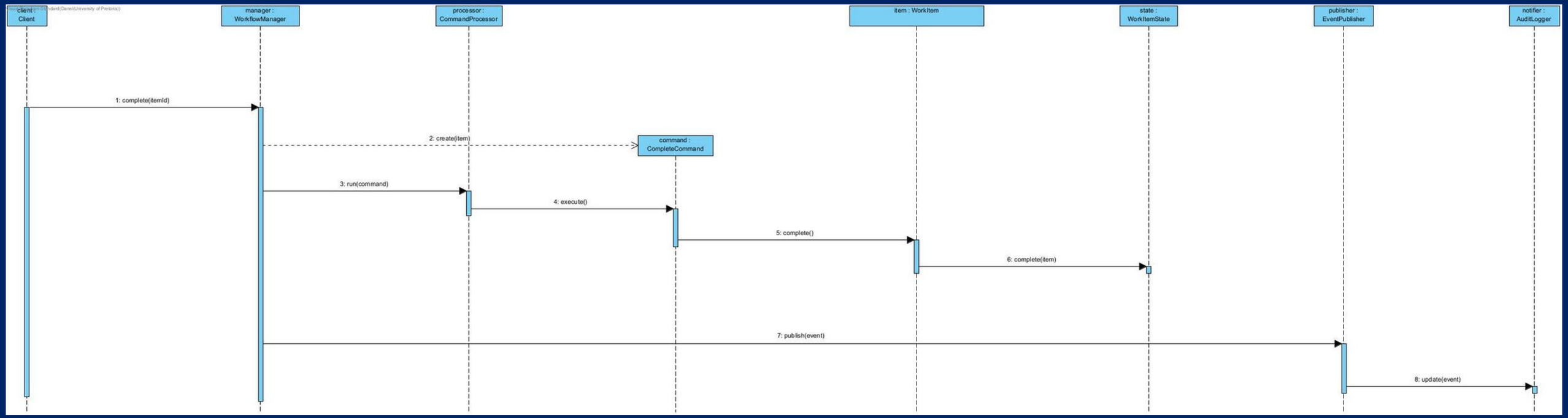
GENERIC WORKFLOW EXAMPLE

- Configurable stages, tasks and dependencies
- Parallel execution of independent tasks
- Conditional decisions and approval processes
- Automated progression through the workflow

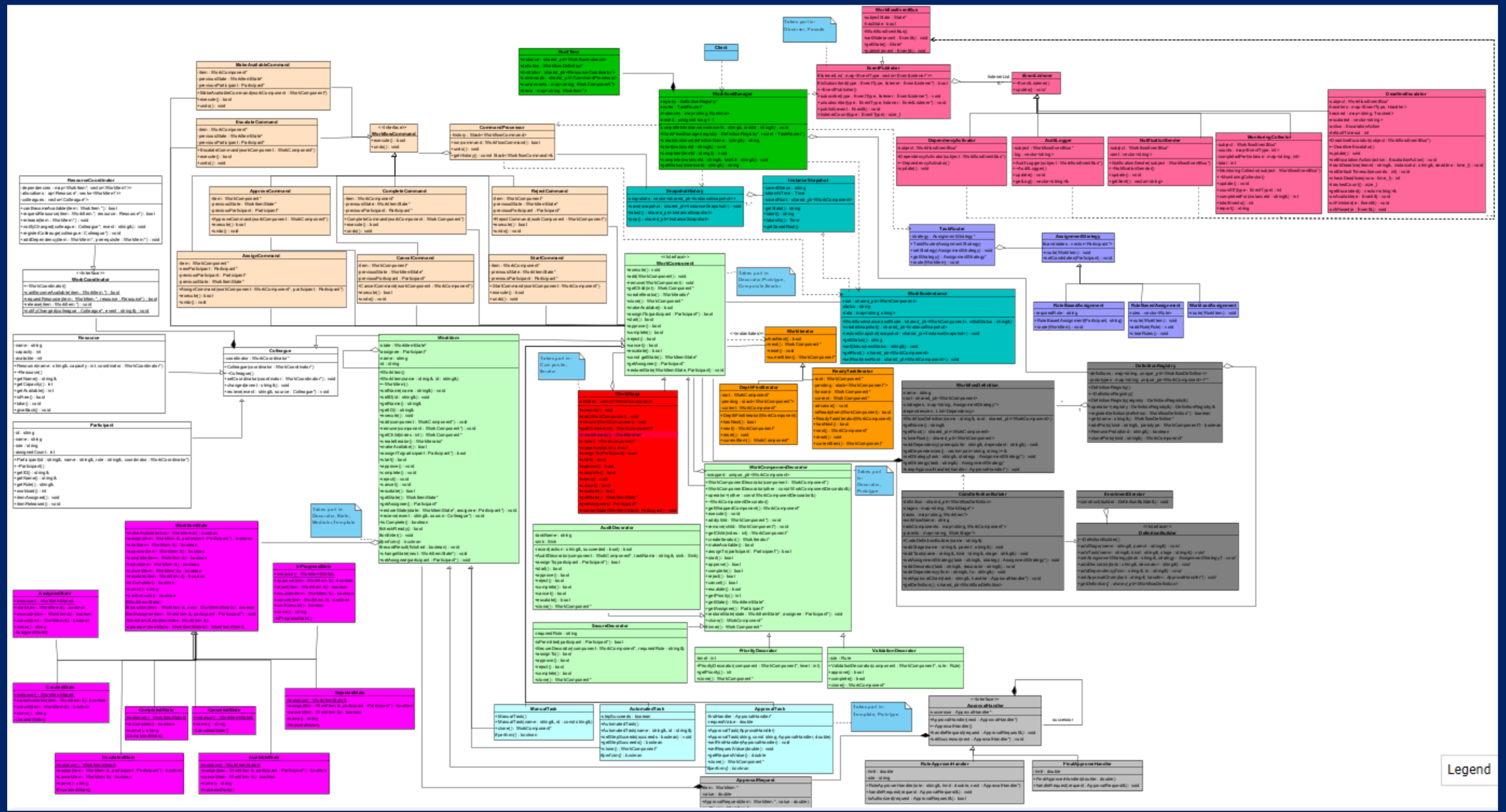
SEQUENCE DIAGRAM Create & Start



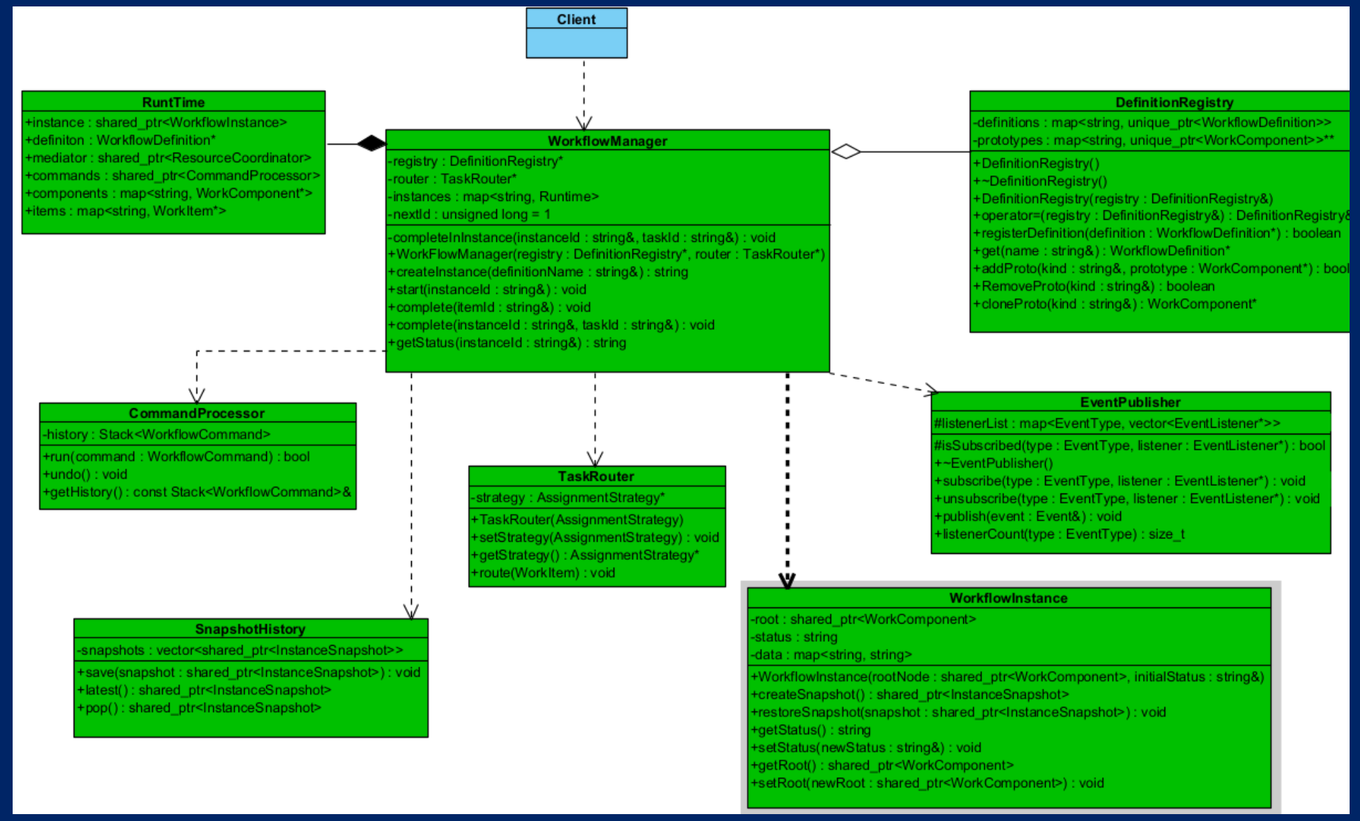
SEQUENCE DIAGRAM Complete work item



CLASS DIAGRAM



FACADE



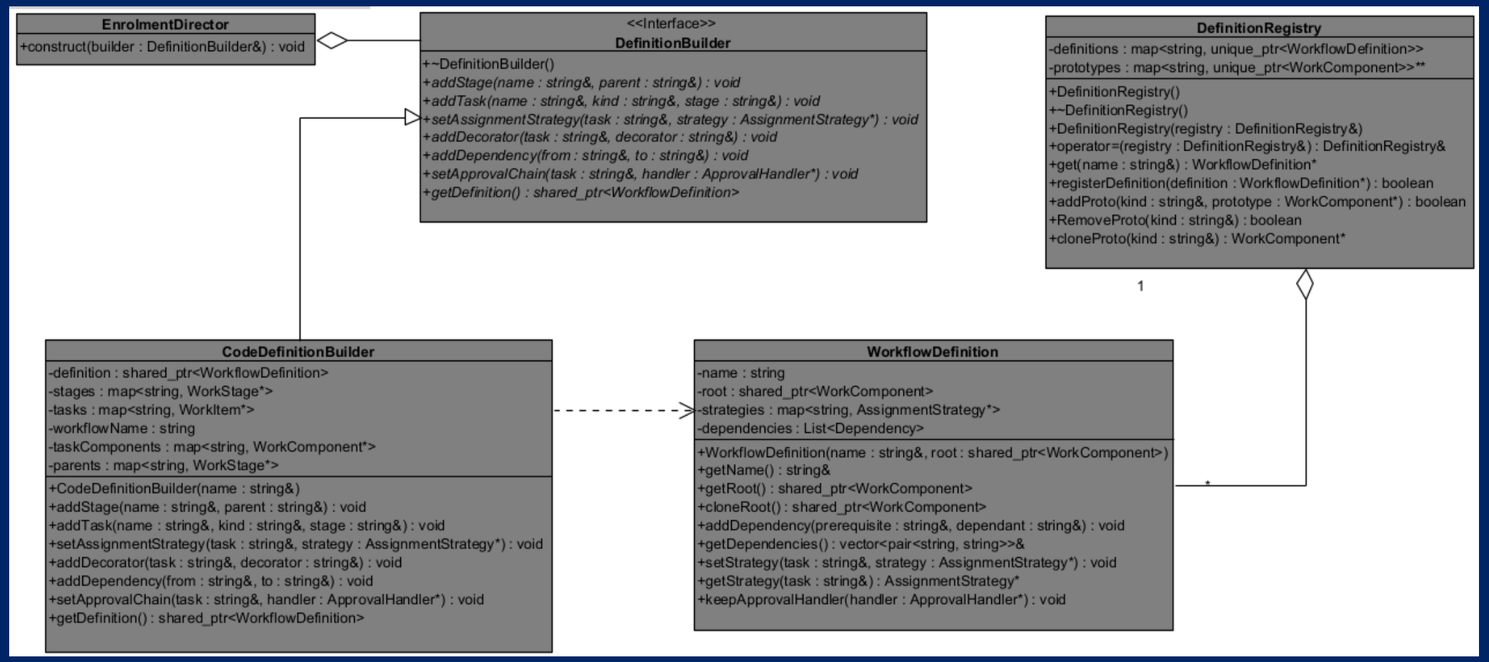
- **Classes Involved:** WorkflowManager, DefinitionRegistry, WorkflowInstance, TaskRouter, EventPublisher, CommandProcessor, SnapshotHistory

- **The problem:** A workflow system has many subsystems (definitions, instances, etc). If clients called each one directly, they would need to know the engine's internal structure.

- **How does it solve the problem:** One entry point offers simple operations to create, start and check a workflow, and coordinates the subsystems behind it. Clients never need to know the internal structure or the order of calls.

- **Generic vs Scenario:** Generically, the subsystems behind the entry point can be anything and can change freely. In the university scenario, the enrolment process only ever talks to this one entry point to create an application, start it and check its status.

BUILDER



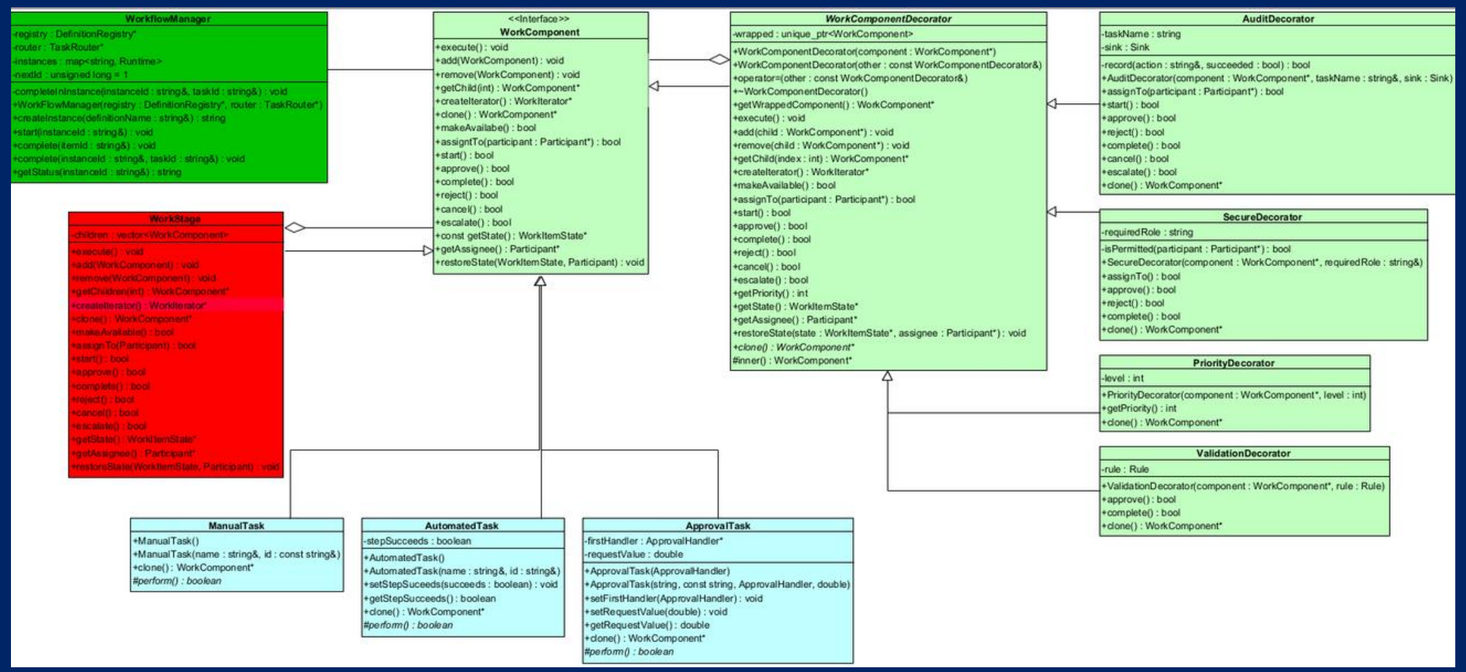
- **Classes Involved:** EnrolmentDirector, DefinitionBuilder, CodeDefinitionBuilder, WorkflowDefiniton, DefinitionRegistry

- **The problem:** A workflow definition is a complex nested structure of stages, tasks, rules, and dependencies. Constructing it with one large constructor is error-prone.

- **How does it solve the problem:** The definition is assembled step by step and returned as a finished product, so construction is kept separate from the structure itself.

- **Generic vs Scenario:** Generically, the builder can produce any workflow from any source. In the university scenario, all the enrolment-specific knowledge (admission, fees, registration, their tasks and approval routes) lives in one place that drives the builder.

PROTOTYPE



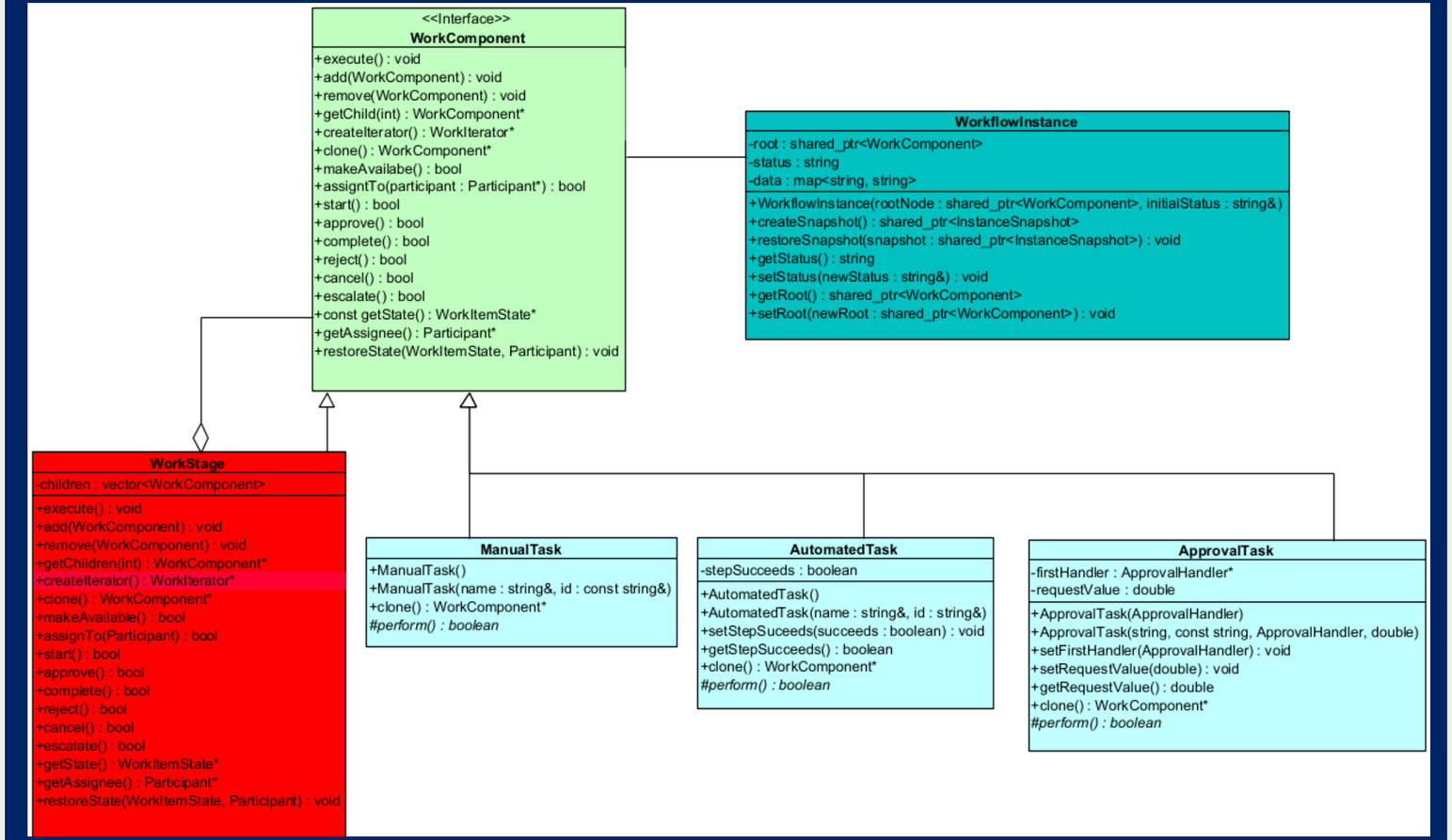
- **Classes Involved:** WorkflowManager, WorkComponent, WorkStage, ManualTask, AutomatedTask, ApprovalTask, WorkComponentDecorator, AuditorDecorator, SecureDecorator, PriorityDecorator, ValidationDecorator

- **The problem:** A change to one running instance should not alter every other instance created from the same definition. Each instance created from a definition needs its own copy of the structure, data, and progress.

- **How does it solve the problem:** Each new instance is a deep copy of a stored definition, so no stage or task is shared between instances. Changes to one instance never affect another or the original.

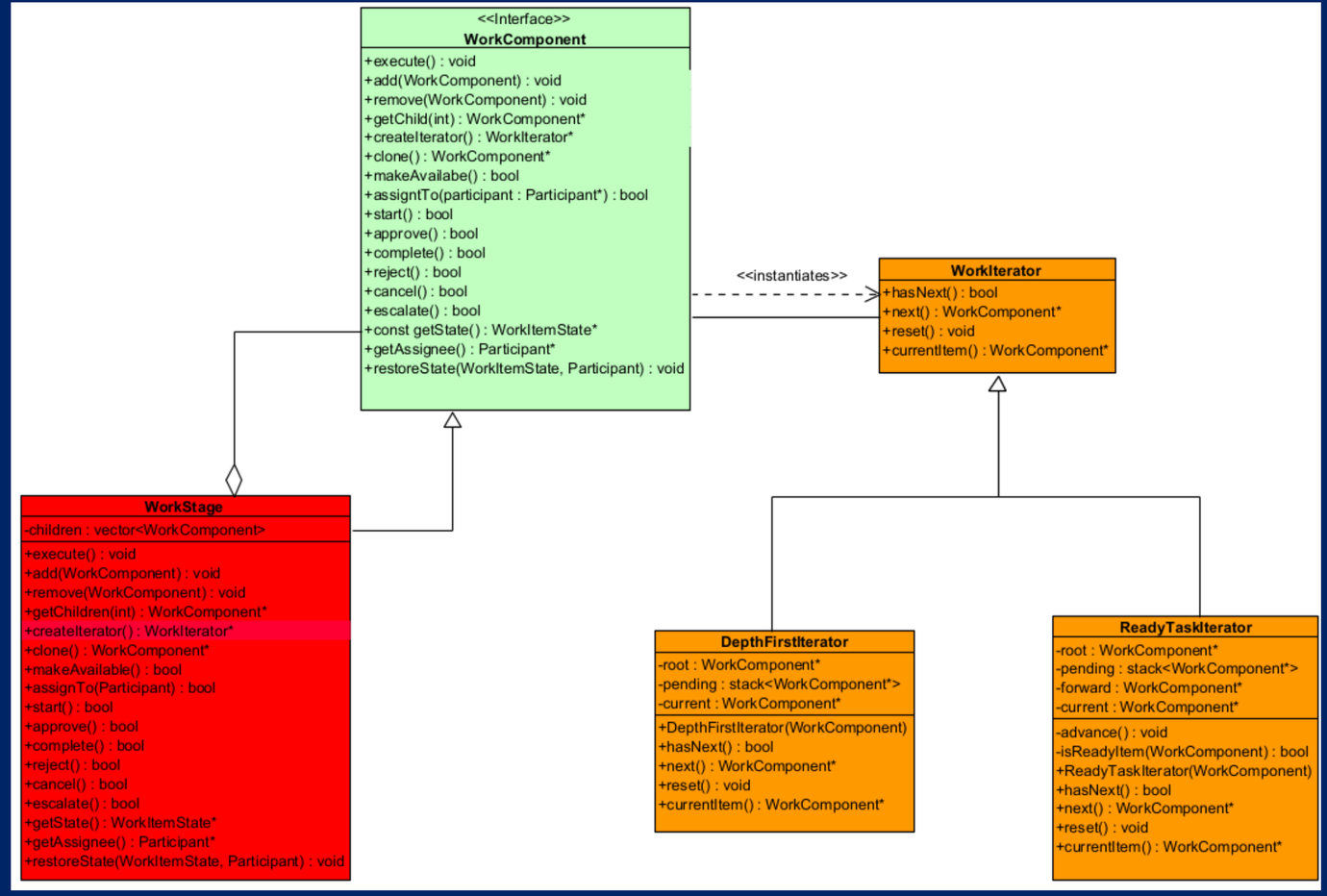
- **Generic vs Scenario:** Generically, any stored definition can be cloned to create instances. In the university scenario, every applicant gets their own copy of the enrolment process.

COMPOSITE



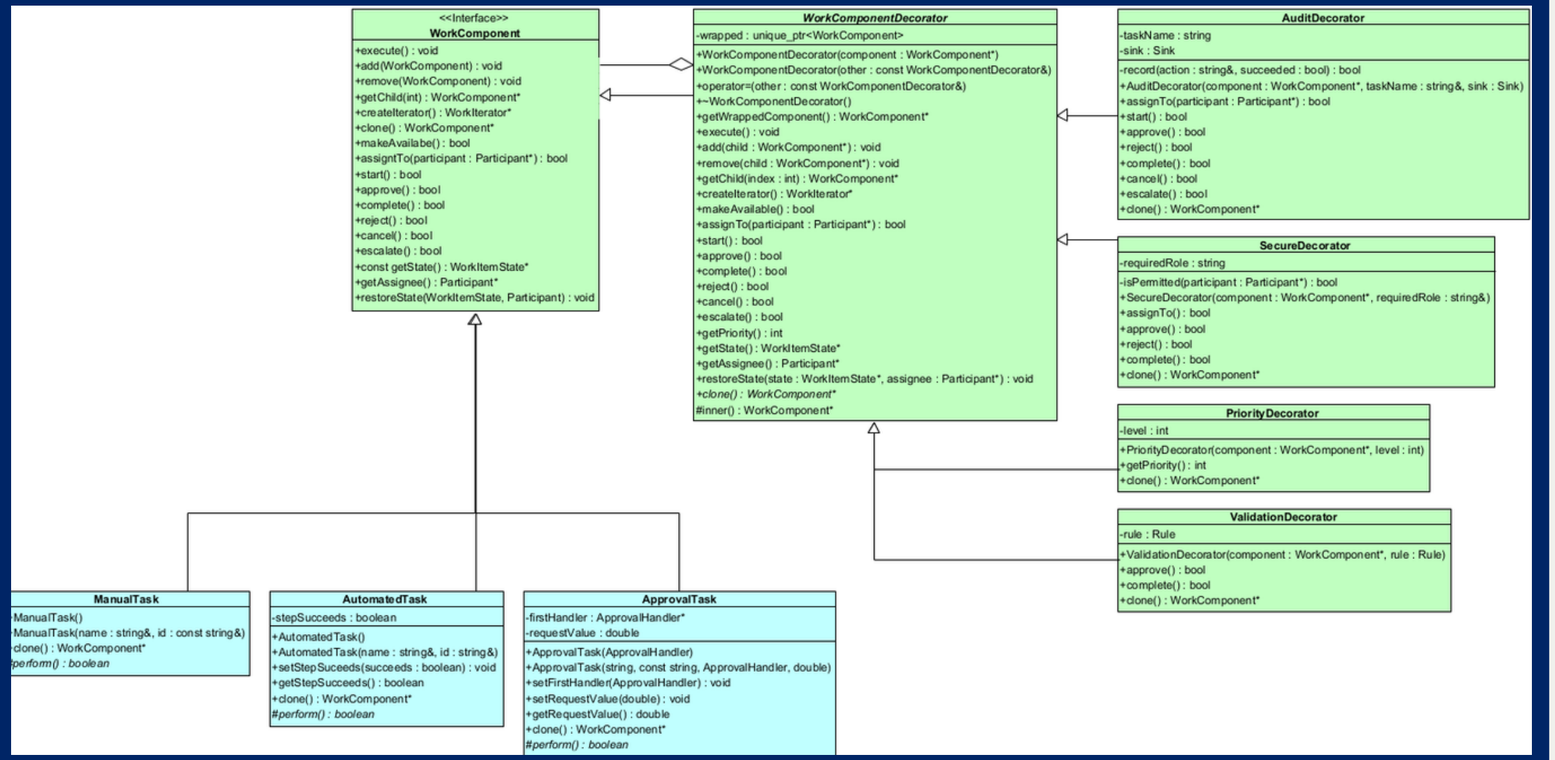
- **Classes Involved:** WorkflowInstance, WorkComponent, WorkStage, ManualTask, AutomatedTask, ApprovalTask
- **The problem:** A workflow contains stages, which contain individual work items and sub-processes, which needs to be structured accordingly.
- **How does it solve the problem:** Single tasks and groups share one interface, so the system can start, complete or query a task or a whole sub-process in the same way.
- **Generic vs Scenario:** Generically, the depth of nesting and whether a group runs in sequence or in parallel differ per process. In the university scenario, the enrolment process contains admission, fees and registration, each containing its own tasks.

ITERATOR



- **Classes Involved:** WorkComponent, WorkStage, WorkIterator, DepthFirstIterator, ReadyTaskIterator
- **The problem:** Parts of the system need to go through the work in a workflow, such as finding the next available item or showing progress, but the work is stored as a nested tree structure.
- **How does it solve the problem:** Clients step through the work with a standard interface without seeing how groups store their children.
- **Generic vs Scenario:** Generically, the traversal order and the storage of the groups can vary. In the university scenario, one traversal finds the application tasks that are available to be picked up, and another walks the whole enrolment process to show an applicant's progress.

DECORATOR



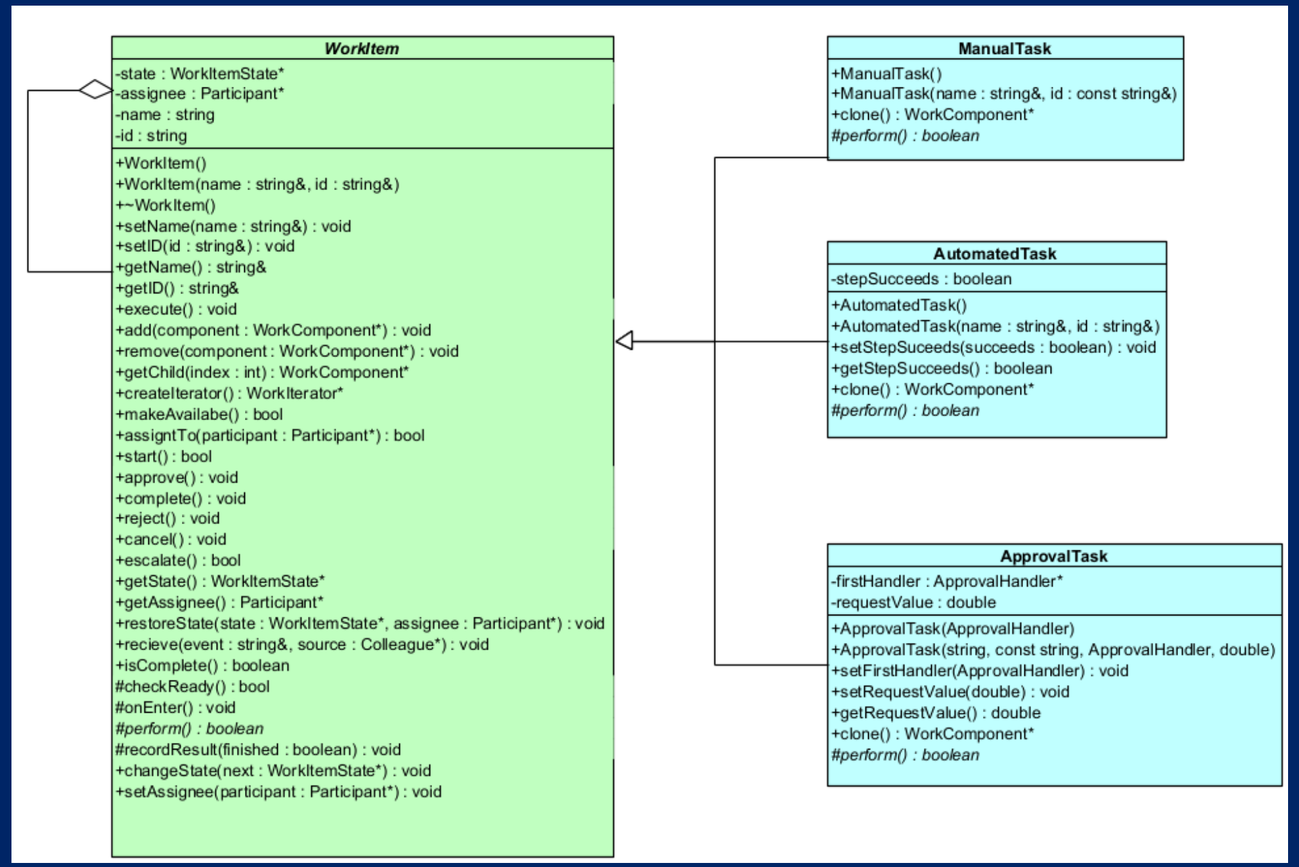
- **Classes Involved:** WorkComponent, ManualTask, AutomatedTask, ApprovalTask, WorkComponentDecorator, AuditDecorator, SecureDecorator, PriorityDecorator, ValidationDecorator

- **The problem:** Different tasks need different additional functionality, such as validation, priority handling, auditing, notifications, or security, and often multiple at once.

- **How does it solve the problem:** A task is wrapped in a decorator that has the same interface. Behaviours can be combined freely, and a new one is just one new decorator with no change to existing tasks.

- **Generic vs Scenario:** Generically, any combination of behaviours can differ per task. In the university scenario, a task like paying a fee might be audited, while confirming registration might notify the applicant.

TEMPLATE



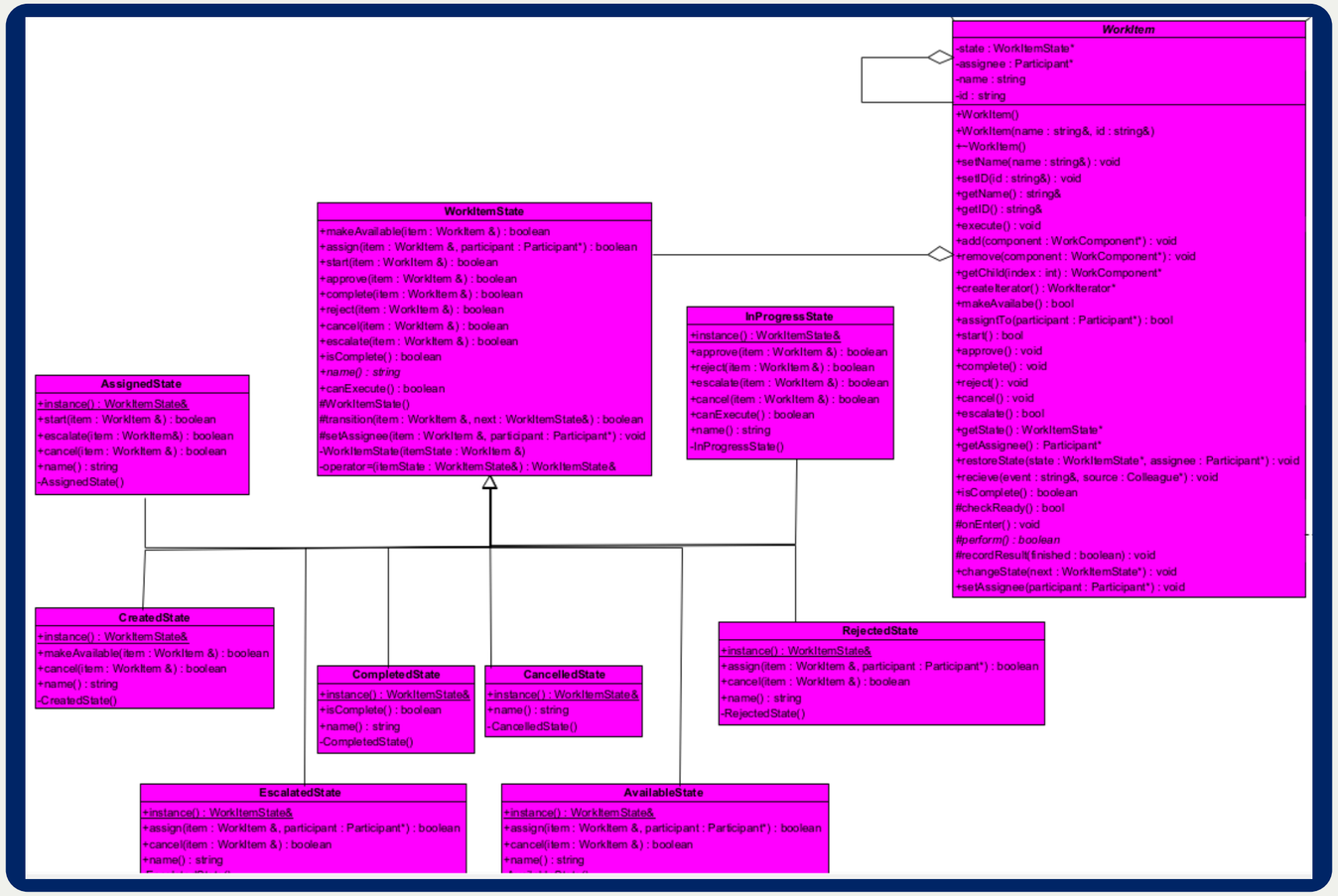
- **Classes Involved:** WorkItem, ManualTask, AutomatedTask, ApprovalTask

- **The problem:** Every workflow instance runs through the same overall execution steps, but the details of some steps differ per process.

- **How does it solve the problem:** The base class fixes the execution order once, and subclasses override only the step that does the actual work.

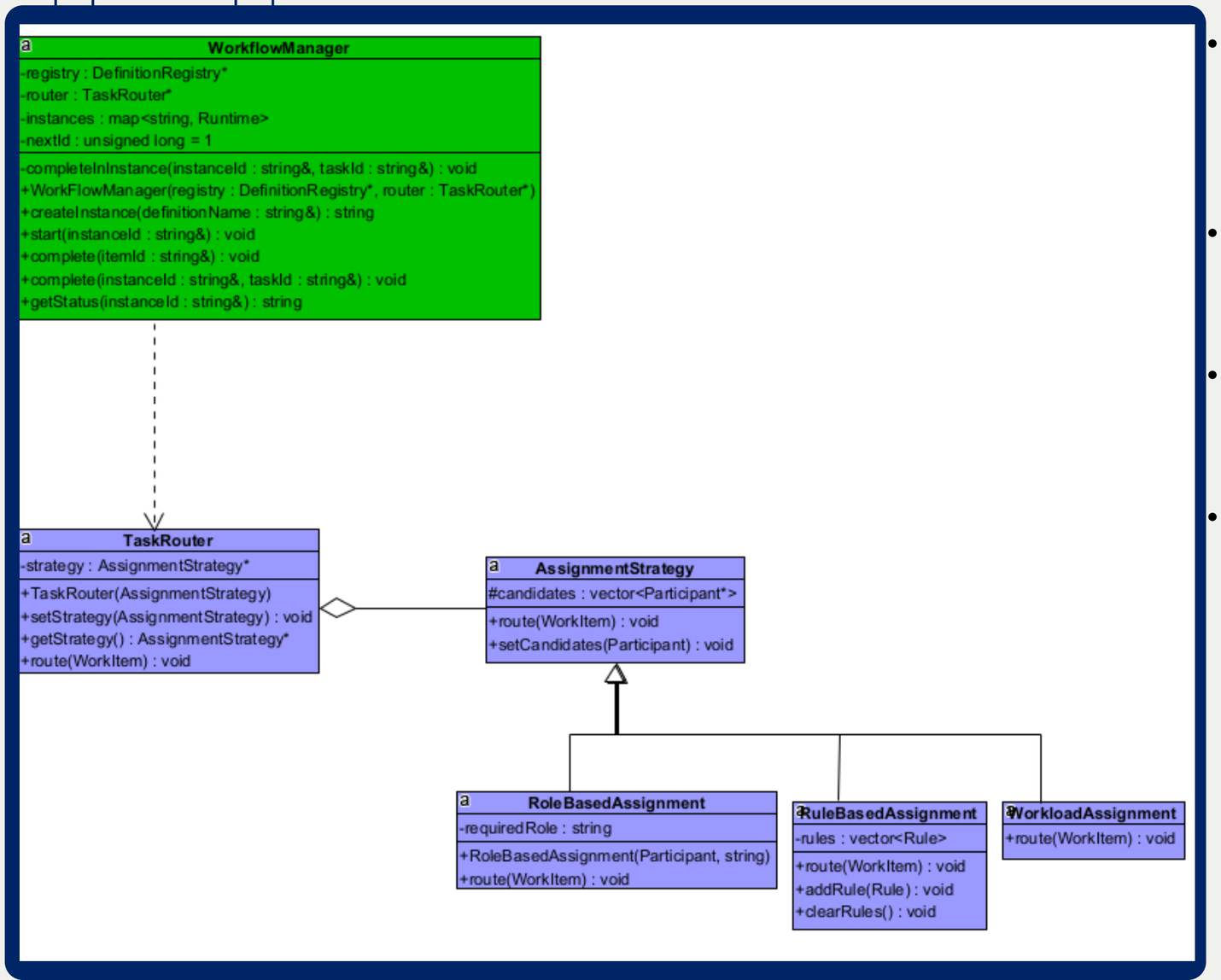
- **Generic vs Scenario:** Generically, the step that differs per task type can be anything. In the university scenario, a manual task such as assessing an application waits for a staff member, an automated task such as charging a fee calls the payment service, and an approval task sends the application up the approval chain.

STATE



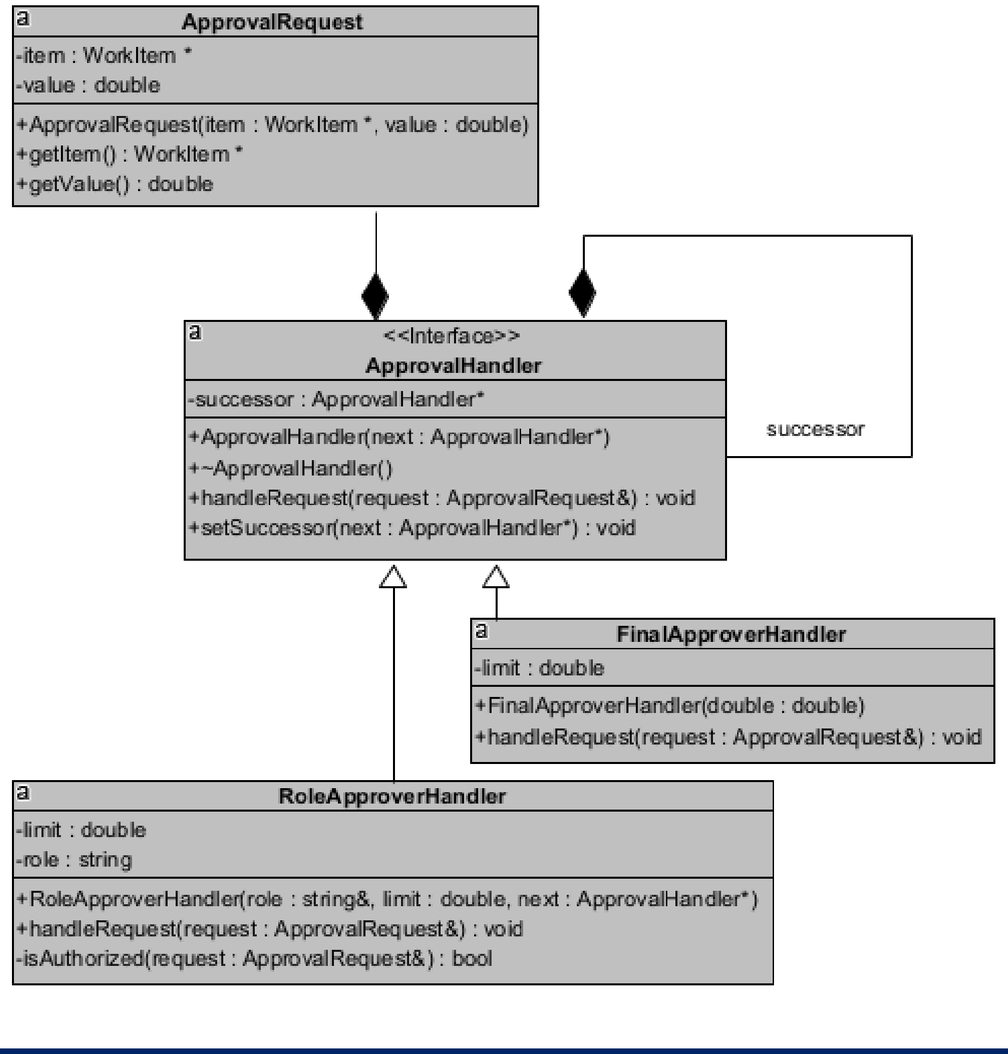
- **Classes Involved:** WorkItem, WorkItemState, AssignedState, CreatedState, CompletedState, CancelledState, EscalatedState, AvailableState, InProgressState, RejectedState.
- **The problem:** What a work item is allowed to do depends on its current stage in its lifecycle.
- **How does it solve the problem:** Each stage decides for itself which actions are valid and what comes next. This avoids a chain of conditions in every operation, and a new stage is added without touching the existing ones.
- **Generic vs Scenario:** Generically, there are basic concrete states such as Completed, Started and Escalated. However, when taking the scenario into account, these concrete states may change, such as replacing the Completed state with an Accepted state for the applications, which can be cancelled, unlike in the generic structure.

STRATEGY



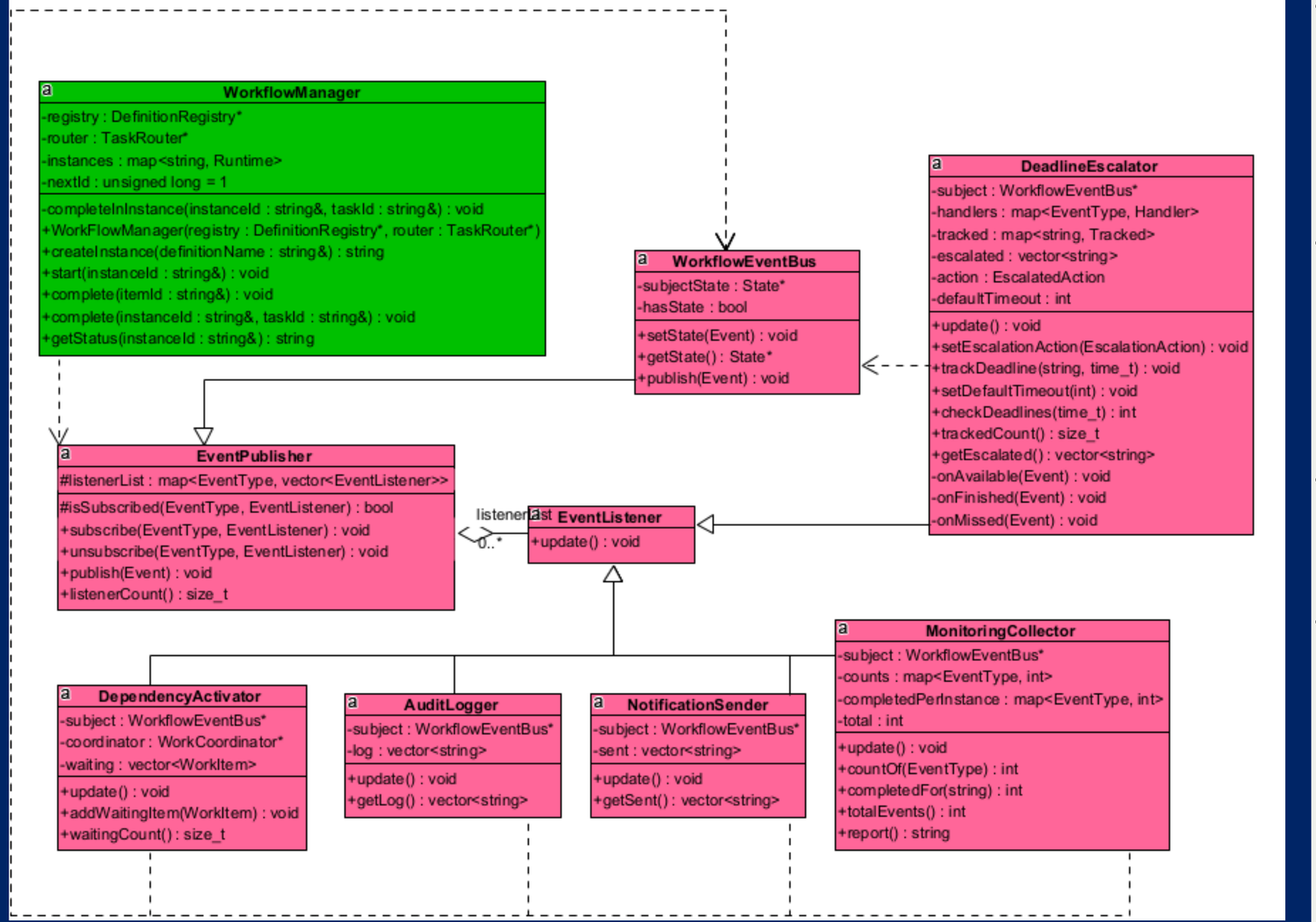
- **Classes Involved:** WorkflowManager(Facade), TaskRouter, AssignmentStrategy, RoleBasedAssignment, RuleBasedAssignment, WorkloadAssignment.
- **The problem:** Different strategies to do the same thing depending on certain factors like roles and rules.
- **How does it solve the problem:** Using the Strategy pattern allows us to assign tasks based on a predefined factor.
- **Generic vs Scenario:** Our generic implementation uses Role, Rule, and workload as predefined factors for the concrete stages, but the concrete stages may differ in specific scenarios i.e. qualification in a hospital setting.

CHAIN OF RESPONSIBILITY



- **Classes Involved:** ApprovalRequest, ApprovalHandler, RoleApproverHandler, FinalApproverHandler.
- **The problem:** Approval of a task may require the task to be escalated higher up.
- **How does it solve the problem:** Chain of responsibility is easily able to fix this as if the first handle(RoleApproveHandler) can’t handle the task, it escalates it to the next handler.
- **Generic vs Scenario:** The generic implementations use predetermined placeholder names and limit values, whereas in scenario-specific implementation, the limit would be a more meaningful value and the classes would be named after who is handeling the task instead of a generic name.

OBSERVER



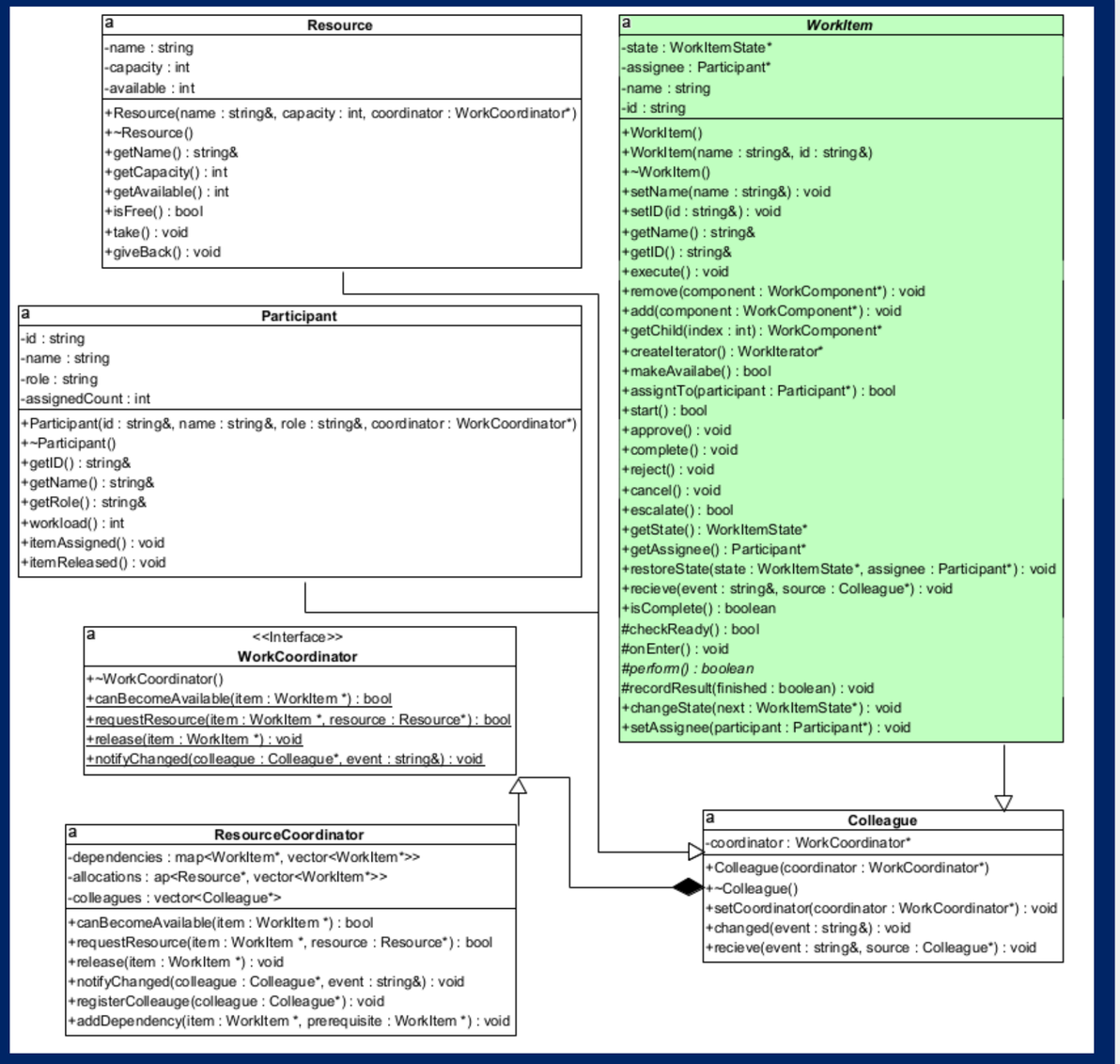
- **Classes Involved:** WorkFlowManager(Facade), WorkflowEventBus, DeadlineEscalator, EventPublisher, EventListener, DependencyActivator, AuditLogger, NotificationSender, MonitoringCollector.

- **The problem:** Completing one activity may make another available, an external event may trigger new work, and several parts of the system may need to react to the same event.

- **How does it solve the problem:** Observer allows us to notify activities when events occur.

- **Generic vs Scenario:** The generic implementation relies on general events, such as a deadline. However scenario specific cases may have different events and reactions to events.

MEDIATOR



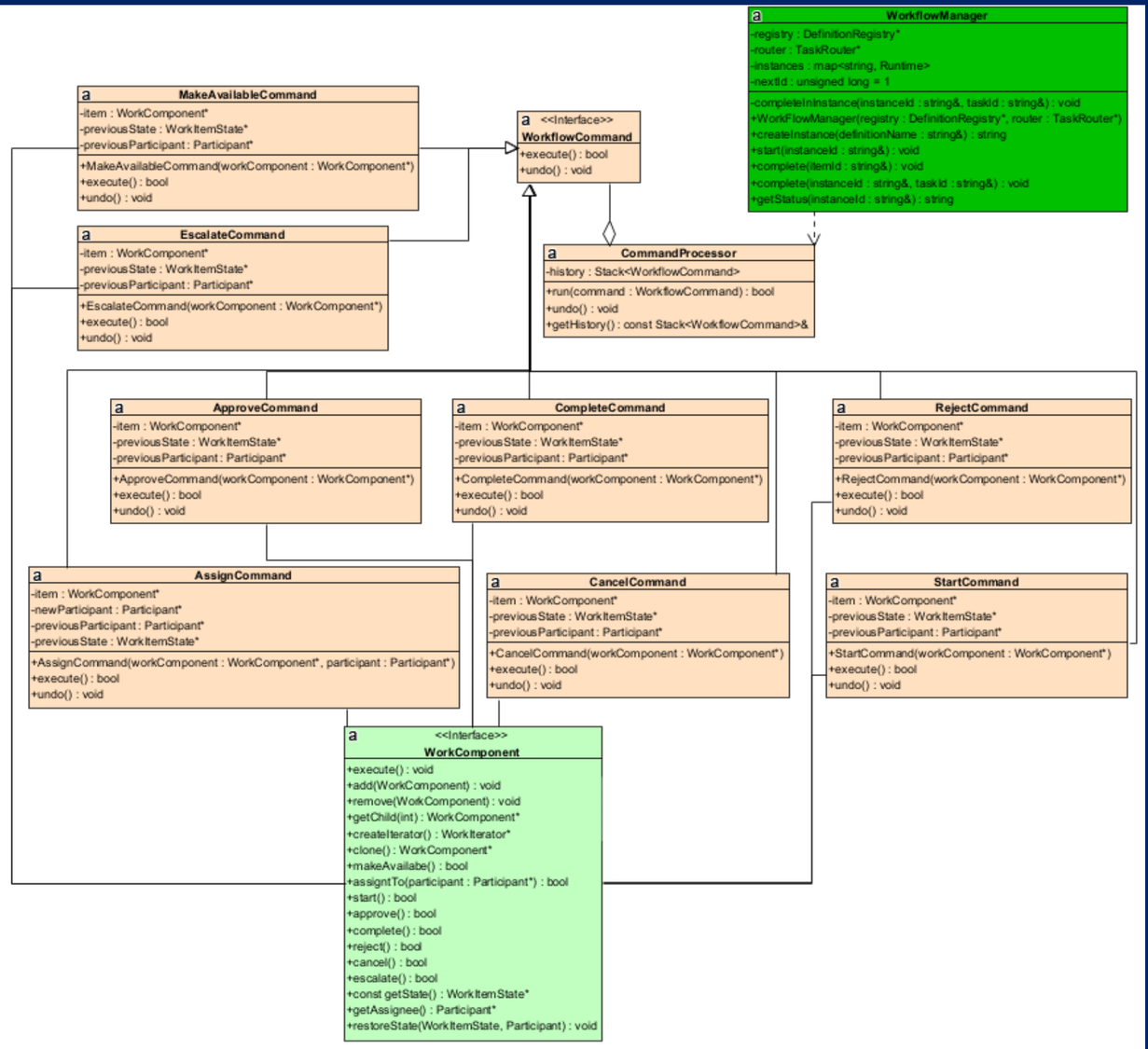
- **Classes Involved:** WorkItem, Resource, Participant, Colleague, WorkCoordinator, ResourceCoordinator.

- **The problem:** Work items, participants, and resources depend on each other i.e. one task waits for another, or participants share a resource.

- **How does it solve the problem:** Mediator is able to notify relevant classes when actions are completed instead of telling every class which allows the dependent objects to carry on.

- **Generic vs Scenario:** Genericly we have our work item, resource and participant that get notified however in a scenario situation the classes the mediator notifies may differ in functionality and name.

COMMAND



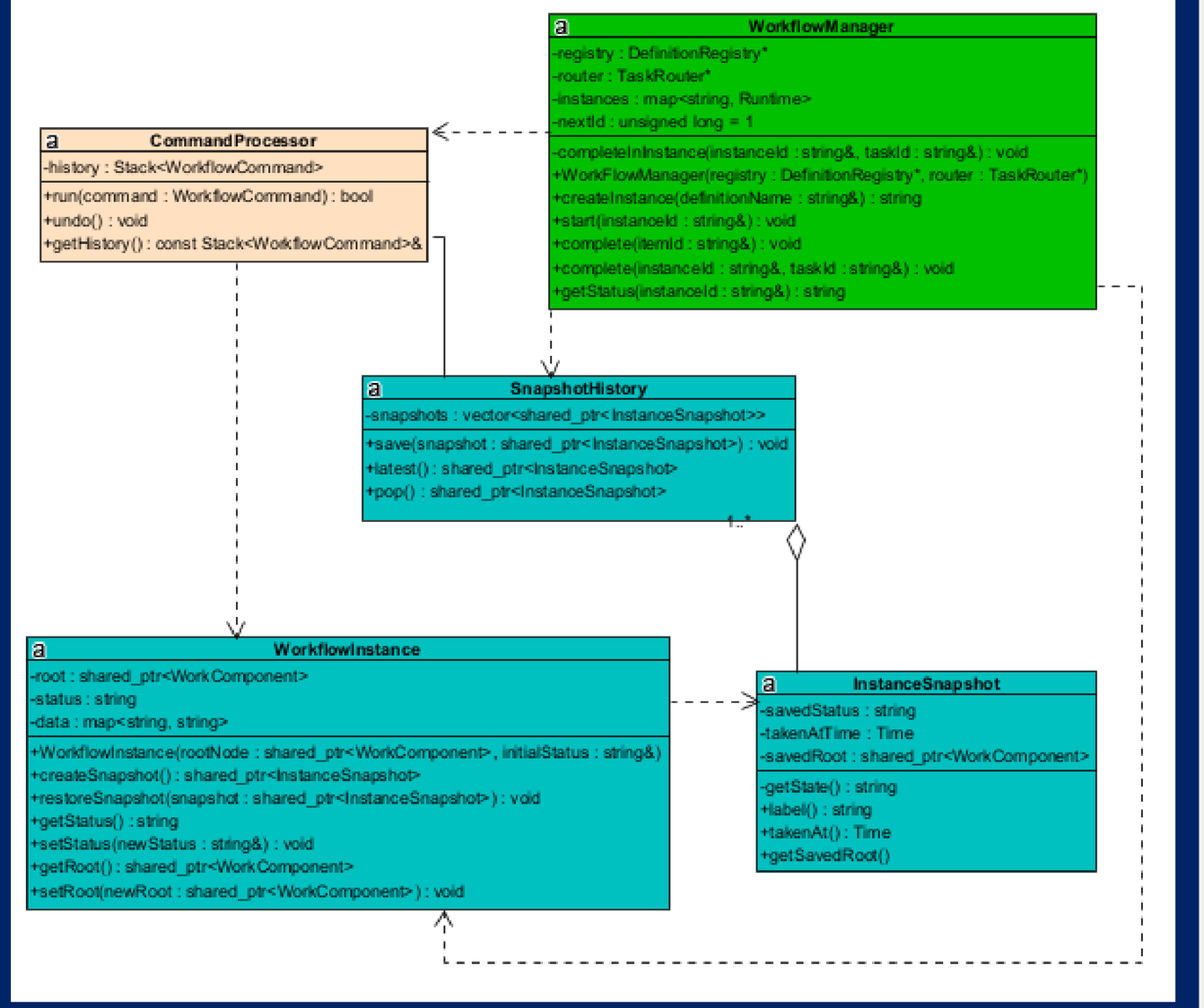
- **Classes Involved:** WorkflowManager(Facade), WorkflowCommand, MakeAvailableCommand, EscalateCommand, CommandProcessor, ApproveCommand, CompleteCommand, RejectCommand, AssignCommand, CancelCommand, StartCommand, WorkComponent.

- **The problem:** Actions must be retained as history and undone if needed.

- **How does it solve the problem:** Command is able to use the inverse to undo certain commands and if needed use the snapshot from momento to preform undo.

- **Generic vs Scenario:** The commands available as well as the way undo() is done will differ per scenario.

MEMENTO



- **Classes Involved:** WorkflowManager(Facade), CommandProcessor, SnapshotHistory, InstanceSnapshot, WorkflowInstance.
- **The problem:** If the system crashes a rollback needs to be available to restore the system to a previous state.
- **How does it solve the problem:** The momento stores a snapshot of the system before a risky action is going to take place and can restore it if needed.
- **Generic vs Scenario:** The difference in the generic and scenarios is when the system decides to take a snapshot of the system.

FRONTEND

TIVIDY Web Interface

Runs on the real engine: a local C++11 server links the TIVIDY engine directly, so every click calls real engine classes. Nothing is simulated in the browser.

One generic engine, any industry: University admission and Patient booking are templates in the same six-step workflow, and new industries can be modelled live without changing engine code.

Design patterns in action: after each action, a live panel shows which GoF patterns and classes took part, and a dedicated tab separates the reusable engine from scenario-specific data.

LIVE DEMO

And Questions?

GitHub

Commit History: We maintained a clear and consistent commit history with concise descriptions to track changes, debug issues and merge code updates incrementally.

Branching & Pull Requests: We used feature branches for every different pattern and pull requests to ensure smooth integration into the main branch.

Automated Documentation: Integrated Doxygen commenting syntax across all C++ headers and source files to automatically generate comprehensive class and architecture documentation.

Conclusion

The TIVIDY project successfuly implemented a reusable workflow management system for defining, running and monitoring a generic workflow that can be used as a university enrolment system, a healthcare booking system etc.
By applying design patterns such as State, Observer, Composite, Chain of Responsibility, Strategy and Command, we supported work item life cycles, role-based routing, multi-level approval and escalation, event-driven monitoring, undo and rollback.
This project included UML diagrams, C++ 11 code and throurough documentation (that can be found in the README), fulfilling the technical goals required and preparing us for future projects to come.